Leif Olson
Saul Duran
ECS-152A

Project 1.1 - iPerf

Project Files Submitted:

- **iPerf**
  - udp_client[Leif_Good-Olson]_[921489807]_[Sual_LastName]_[Saul_StudentID].py
  - udp_server[Leif_Good-Olson]_[921489807]_[Sual_LastName]_[Saul_StudentID].py
- **iPerf_ChatGPT-Assisted**
  - udp_client[Leif_Good-Olson]_[921489807]_[Sual_LastName]_[Saul_StudentID].py
  - udp_server[Leif_Good-Olson]_[921489807]_[Sual_LastName]_[Saul_StudentID].py

****Please consider the "ChatGPT-Assisted" files to be our final submission - I was confused about what exactly to include in the submission so this is how I formatted it.**

[Link to ChatGPT chat session]

1. **How are we supposed to run your code?**
   a. You can run the code by navigating to the directory "iPerf_ChatGPT-Assisted", and running the server and then the client applications (be sure to run the server first so the client has something to connect to) using "python udp_server" and "python udp_client" in a command line interface. Both will remain open for a short while but will time out if no interaction is detected, giving you some time to enter the desired payload size into the terminal running the client. Then, the client will output the information returned by the server.
2. **Implementation Details**
   a. To explain simply, using the socket module, a socket is created and bound to a default host and port (for server and client). The server then listens for incoming connections, at which point the client (upon input of payload size in megabytes by the user) will send that payload

size to the server so that it knows how much data to expect. Simple calculations are run to determine the expected number of packets based on payload size and packet size. From here, the client sends its payload in chunks, with the server accepting them as quickly as it can. At the end, information on how long the transmission took from start to finish as well as the number of bytes received are sent back to the client to be output for viewing.

3. **LLM Section**
    a. I mainly used a LLM (ChatGPT specifically) to see if it had any suggestions for improvements. One such improvement was simplifying the code, replacing functions that sent and received to/from specific IPs with those that functioned based on the connection to the socket. I also improved the timeouts based on its suggestions so that the sockets would simply time out when not receiving any information after some time.
    b. Finally, I tried to use ChatGPT for debugging purposes, as the programs would randomly not work (due to packet loss). It offered many suggestions that ultimately did not help, but in the end helped me to determine that increasing the socket buffer size would prevent packet loss and significantly increase data transmission, solving all remaining issues. I would not have known about the setsockopt() function otherwise.